harveysingh.xyz
harveylx
Harvey Singh

Harvey Singh

harvirjaswal@gmail.com
+44-7592 631 425

London, United Kingdom

Last updated on July 5, 2026

Technical Skills

Languages	**C#**, C++, Kotlin, Java, **TypeScript**, JavaScript, Elixir
Backend	**.NET Core**, **Entity Framework**, FastEndpoints, **Web API**, FluentValidation, Dapper, Phoenix, Ecto, Oban
Frontend	**React**, Angular, **React Native**, Phoenix LiveView
Testing	xUnit, **Testcontainers**, Jest
Databases	**PostgreSQL**, MongoDB, SQL Server, DynamoDB, CosmosDB
Systems / Platforms	**GCP**, AWS, Azure, **Kubernetes**, **Docker**, Pulumi, Terraform, **DirectX 11**, ImGui, Win32 API

Work Experience

Feb '26 - Present

Senior Software Engineer - Duffel *Travel Fintech Platform*

- Built Duffel's **profit-share system** in **Elixir**: tiered pricing calculations with FX normalisation, **Ecto** schemas with optimistic locking, 10+ **Phoenix LiveView** admin pages, CLI tooling, and 5 production database migrations
- Created the supplier commission estimates pipeline using a contract pattern with **Elixir behaviours**, **Oban** background jobs with fan-out processing, and upsert logic for idempotent ingestion
- Added Conferma virtual card reconciliation to Jet (**Phoenix LiveView**) across 5 umbrella apps: case management with comments, manual reconciliation, automatic card ID resolution for refunds, and revenue discrepancy creation
- Wrote CSV commission parsers for three hotel suppliers (Booking.com, Onyx, Priceline), dealing with UTF-8 BOM encoding, inconsistent field casing, truncated timestamps, and nullable fields
- Ran zero-downtime schema migrations for the tokenised cards feature: added the new column, backfilled existing data, then dropped the legacy column across 9 umbrella apps
- Fixed a production timeout in draft payout creation by optimising the **Ecto** query, and built 5 CLI tasks so the payments team could self-serve profit-share runs and commission corrections

Nov '25 - Present

Founder / CTO - Wavara *Cycle-Aware Weight Tracking App*

- Built a full-stack **polyglot monorepo**: a **React Native** mobile app (**Expo SDK 54**, **TypeScript**, **React 19**) paired with a **C# .NET 10** Web API, shipping to **iOS** and **Android**
- Used **SQLite** for offline weight and cycle tracking with a migration-based schema and repository pattern, designed for future backend sync
- Built the API as a vertical slice using **FastEndpoints** (REPR pattern) with **PostgreSQL 17**, **Entity Framework Core 10**, **JWT bearer authentication**, email verification flows, and rate-limited anonymous endpoints
- Wrote a custom charting library using **D3** and **React Native SVG**, with cycle-phase-coloured backgrounds, selectable date ranges, and tap-to-inspect tooltips
- Created the design system with **React Native Reanimated** spring and timing animations, **Skia** particle effects, and a warm-dark colour palette tied to cycle phases
- Set up a type-safe API contract workflow: **OpenAPI** spec generated from the .NET API, consumed via **TypeScript** codegen in the mobile app
- Deployed the API to **Hetzner Cloud** (ARM64) with **Docker Compose**, **Caddy** for automatic TLS, and **Resend** for transactional email, running production and dev on a single server
- Configured **EAS Build** pipelines for development, preview, and production profiles with auto-versioning, **Sentry** error tracking, and **PostHog** analytics (consent-gated)
- Wrote tests across both stacks: **Jest** + **React Native Testing Library** for the mobile app; **xUnit** + **Testcontainers** (throwaway PostgreSQL) for API integration tests
- The cycle-phase calculation engine works out the user's current menstrual phase from period start dates and configurable cycle length, which drives the charts, daily guidance, and visual hierarchy throughout the app

Jun '24 - Present

Software Engineer - Propelle Ltd. *Wealth Management Platform*

- Architected and maintained 13 **microservices** implementing **Domain-Driven Design (DDD)** principles with rich aggregates, value objects, and domain events for complex financial products (ISAs, GIAs, Junior ISAs), handling wealth management, trading, compliance, and account management for a production financial services platform serving mobile applications on **Google Cloud Platform (GCP)**, building resilient, scalable **distributed systems** using **.NET 8, PostgreSQL, Entity Framework Core**, and GCP services (**Cloud Run**, Cloud SQL, Cloud Tasks, Secret Manager)
- Designed and implemented an **event-driven architecture** using **Google Cloud Pub/Sub** for asynchronous communication between services, ensuring eventual consistency across distributed transactions with comprehensive error handling, retry mechanisms, and circuit breakers to achieve 99.9% uptime for critical financial operations
- Engineered complex domain models following DDD principles for financial products with sophisticated fee processing system spanning multiple services, implementing automated monthly calculations with daily compounding, discrepancy detection, and multi-tier rate hierarchies (member rates, fund rates, overrides), coordinating asset selling across portfolios with tolerance-based adjustment mechanisms
- Built JISA account management feature applying advanced domain modelling techniques for complex parent-child relationship structures, supporting multiple ownership roles (registered contact, beneficiary) with proper authorisation and compliance checks, leading major data migration from single-ownership to many-to-many ownership model using **Entity Framework Core** code-first migrations with proper indexing and constraints
- Designed mobile-first **RESTful APIs** using **FastEndpoints** with versioning strategies, backward compatibility, and mobile-optimised patterns (payload minimisation, batch operations, offline support), implementing comprehensive authentication and authorisation using **JWT Bearer tokens** with OpenIddict, supporting role-based access control across 13 services with robust API validation using FluentValidation
- Built compliance monitoring service implementing regulatory requirements, tracking deposits, withdrawals, and transactions with automated alerting for suspicious activities and threshold breaches, implementing **KYC (Know Your Customer)** workflows with document verification, risk assessment, and regulatory reporting, alongside multi-stage account holder onboarding system with third-party identity verification integration
- Integrated multiple third-party APIs including WealthKernel broker API for investment account creation, trading execution, settlement tracking, regulatory reporting and **KYC** verification workflows; Plaid (**Open Banking**) payment integration for secure bank account linking and deposit processing with webhook handling; RevenueCat for mobile subscription management and membership tier synchronisation
- Engineered automated trading system handling order execution, matching, settlement, and charge creation through WealthKernel's broker API, building portfolio rebalancing engine with sell-down and buy-up order orchestration, implementing complex financial algorithms including daily compounding fee calculations, market data handling, available units calculations, and withdrawal/deposit processing with compliance checks for regulatory limits (ISA allowances, JISA limits) and automated reconciliation
- Identified and resolved a critical double-processing bug in the shared **outbox pattern** library that had persisted for 3 years across the platform, conducting root cause analysis that revealed Entity Framework Core's optimistic concurrency model allowed race conditions during message processing, implementing a fix using **pessimistic locking** with raw SQL queries and row-level locks to ensure exactly-once message delivery semantics across all microservices
- Established comprehensive testing strategy following the testing pyramid with **xUnit, Testcontainers**, and WebApplicationFactory for **integration testing**, achieving high code coverage by building fake implementations for external services (WealthKernel, Plaid, RevenueCat, Blend) enabling fast, reliable integration tests, and implementing test data generation using AutoFixture and custom builders for complex domain objects
- Implemented **infrastructure as code** using **Pulumi** for GCP resource provisioning (Cloud Run, Cloud SQL, Pub/Sub, VPC, IAM), building **CI/CD pipelines** using **NUKE build** automation and **GitHub Actions** with environment-specific deployment strategies (development, staging, production), configuring containerised deployments with **Docker**, automated database migrations, secret management integration, and comprehensive monitoring with health checks using Google Cloud Diagnostics for observability across the distributed system
- Led platform modernisation initiatives migrating multiple services from .NET 6 to **.NET 8**, ensuring compatibility, performance improvements, and minimal downtime; refactored legacy Controller-based APIs to **FastEndpoints** pattern, improving performance, maintainability, and developer experience whilst maintaining backward compatibility for mobile clients

Oct '23 - Jun '24	**Software Engineer - TAINA Tech Ltd.**	*TAINA Validator*

- Containerised a **.NET 6** Web API and **React** frontend with **Docker**, then set up **Kubernetes** on **Amazon EKS** with **Helm** charts for deployment across environments
- Replaced manual SQL migration scripts with **FluentMigrator**, running migrations automatically on application startup so deployments no longer needed a separate database step
- Refactored the API's dependency injection setup, fixing services registered as transient instead of scoped that were inflating CPU and memory usage, reducing cloud costs and latency
- Found a **SQL Server** connection leak in my first week that was causing query timeouts for customers, traced it to undisposed connections, and fixed it
- Moved the team from end-to-end UI tests to **integration tests** against the API directly, cutting CI pipeline time and reducing flaky failures
- Ran fortnightly mentoring sessions with junior engineers covering clean code practices, SOLID principles, and code review technique

Oct '22 - Sep '23	**Software Engineer - Dojo Ltd.**	*Communications & Identity*
Oct '21 - Aug '22	**Software Engineer - HealthHero Ltd.**	*MedSol PWA / SAPI*
Jul '19 - Oct '21	**Junior Software Engineer - Argus Media Ltd.**	*Direct Workspaces*
Jun '18 - Jun '19	**QA Engineer - ASOS PLC**	*Bag API*
May '17 - Jun '18	**Associate Software Developer in Test - Veritas LLC**	*Information Classifier*

Projects

Mistrunner — Guild Wars 2 Combat Automation

An injected DirectX 11 overlay for Guild Wars 2 that automates combat rotations in real time. Written in **C++17** as a DLL that hooks into the game process via COM vtable hooking. Reads live game state through **reverse-engineered** pointer chains (40+ offsets found via **Ghidra** and **Cheat Engine**), runs rotation logic as a state machine on the game thread, and renders an ImGui HUD on the render thread. Uses double-buffered snapshots for thread-safe state sharing, SEH wrapping to prevent crashes from bad pointers, and **AOB signature scanning** so it survives game patches without manual offset updates.

Education

2012-2017	Bachelor of Science in **Computer Science** **Brunel University**, London	**Award**: Second Upper Class (2:1)

About Me

- Programming and learning new technologies has now become a big part of my life, and I try to keep myself up-to-date with the latest in tech by attending conferences, joining meetup groups, and going to training courses
- I am an avid practitioner of Muay Thai and Boxing. So if you see me pop in with black eyes and several bruises, don't be alarmed!
- I like to keep fit and help others reach their fitness goals. So far I have lost approximately 36kg and have no plans to stop yet! I have taught many friends and family about the best (and healthiest) ways to reach their goals, whether they are trying to gain mass or lose weight, with successful results
- I maintain a personal blog at harveysingh.xyz, built with Astro and deployed on Cloudflare. The site serves as a platform where I document my journey as a developer, share insights about my career, and explore interesting technologies and projects I've worked with along the way

References available upon request